

AD EXP-10

JAVA CODE:

MainActivity.java:

```
package com.example.googlemapgps;

import android.Manifest;
import android.content.pm.PackageManager;
import android.location.Location;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.core.app.ActivityCompat;
import androidx.fragment.app.FragmentActivity;

import com.google.android.gms.location.FusedLocationProviderClient;
import com.google.android.gms.location.LocationCallback;
import com.google.android.gms.location.LocationRequest;
import com.google.android.gms.location.LocationResult;
import com.google.android.gms.location.LocationServices;

import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;

import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.Marker;
import com.google.android.gms.maps.model.MarkerOptions;

public class MainActivity extends FragmentActivity
    implements OnMapReadyCallback {

    private static final int LOCATION_PERMISSION_REQUEST_CODE = 1;

    private GoogleMap mMap;

    private FusedLocationProviderClient fusedLocationClient;

    private LocationRequest locationRequest;

    private LocationCallback locationCallback;

    private Marker currentMarker;

    private Button btnTrack;
```

```

private boolean isTracking = false;

@Override
protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_main);

    fusedLocationClient =
        LocationServices.getFusedLocationProviderClient(this);

    SupportMapFragment mapFragment =
        (SupportMapFragment)
            getSupportFragmentManager()
                .findFragmentById(R.id.map);

    if (mapFragment != null) {
        mapFragment.getMapAsync(this);
    }

    btnTrack = findViewById(R.id.btnTrack);

    createLocationRequest();

    createLocationCallback();

    btnTrack.setOnClickListener(v -> {

        if (!isTracking) {

            startLocationUpdates();

            btnTrack.setText("STOP TRACKING");

            isTracking = true;

        } else {

            stopLocationUpdates();

            btnTrack.setText("START TRACKING");

            isTracking = false;

        }
    });
}

private void createLocationRequest() {

```

```

locationRequest = LocationRequest.create();

locationRequest.setInterval(3000);

locationRequest.setFastestInterval(2000);

locationRequest.setPriority(
    LocationRequest.PRIORITY_HIGH_ACCURACY);
}

private void createLocationCallback() {

    locationCallback = new LocationCallback() {

        @Override
        public void onLocationResult(
            @NonNull LocationResult locationResult) {

            super.onLocationResult(locationResult);

            if (locationResult == null) {
                return;
            }

            for (Location location :
                locationResult.getLocations()) {

                updateMapLocation(location);
            }
        }
    };
}

private void startLocationUpdates() {

    if (ActivityCompat.checkSelfPermission(
        this,
        Manifest.permission.ACCESS_FINE_LOCATION)
        != PackageManager.PERMISSION_GRANTED
        &&
        ActivityCompat.checkSelfPermission(
            this,
            Manifest.permission.ACCESS_COARSE_LOCATION)
        != PackageManager.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(
            this,
            new String[]{
                Manifest.permission.ACCESS_FINE_LOCATION,

```

```

        Manifest.permission.ACCESS_COARSE_LOCATION
    },
    LOCATION_PERMISSION_REQUEST_CODE);

    return;
}

mMap.setMyLocationEnabled(true);

fusedLocationClient.requestLocationUpdates(
    locationRequest,
    locationCallback,
    null
);

Toast.makeText(
    this,
    "Location Tracking Started",
    Toast.LENGTH_SHORT
).show();
}

private void stopLocationUpdates() {

    fusedLocationClient.removeLocationUpdates(locationCallback);

    Toast.makeText(
        this,
        "Location Tracking Stopped",
        Toast.LENGTH_SHORT
    ).show();
}

private void updateMapLocation(Location location) {

    LatLng currentLocation =
        new LatLng(
            location.getLatitude(),
            location.getLongitude());

    if (currentMarker != null) {
        currentMarker.remove();
    }

    currentMarker = mMap.addMarker(
        new MarkerOptions()
            .position(currentLocation)
            .title("Current Location"));

    mMap.animateCamera(

```

```

        CameraUpdateFactory
            .newLatLngZoom(currentLocation, 16));

    Toast.makeText(
        this,
        "Location Updated:\nLat: "
            + location.getLatitude()
            + "\nLng: "
            + location.getLongitude(),
        Toast.LENGTH_SHORT
    ).show();
}

@Override
public void onMapReady(@NonNull GoogleMap googleMap) {

    mMap = googleMap;

    mMap.getUiSettings().setZoomControlsEnabled(true);

    if (ActivityCompat.checkSelfPermission(
        this,
        Manifest.permission.ACCESS_FINE_LOCATION)
        == PackageManager.PERMISSION_GRANTED
        ||
        ActivityCompat.checkSelfPermission(
            this,
            Manifest.permission.ACCESS_COARSE_LOCATION)
            == PackageManager.PERMISSION_GRANTED) {

        mMap.setMyLocationEnabled(true);
    }
}

@Override
public void onRequestPermissionsResult(
    int requestCode,
    @NonNull String[] permissions,
    @NonNull int[] grantResults) {

    super.onRequestPermissionsResult(
        requestCode,
        permissions,
        grantResults);

    if (requestCode
        == LOCATION_PERMISSION_REQUEST_CODE) {

        if (grantResults.length > 0
            &&

```

```

        grantResults[0]
            == PackageManager.PERMISSION_GRANTED) {

            startLocationUpdates();

        } else {

            Toast.makeText(
                this,
                "Location Permission Denied",
                Toast.LENGTH_SHORT
            ).show();
        }
    }
}

@Override
protected void onPause() {

    super.onPause();

    if (isTracking) {
        stopLocationUpdates();
    }
}
}

```

XML CODE:

```

<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <fragment
        android:id="@+id/map"
        android:name="com.google.android.gms.maps.SupportMapFragment"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <Button
        android:id="@+id/btnTrack"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="START TRACKING"
        android:textStyle="bold"
        android:textColor="@android:color/white"

```

```
android:backgroundTint="#2196F3"  
android:padding="16dp" />
```

```
</LinearLayout>
```

O/P:

